Serverless Order Notification & Processing System using AWS SNS, SQS & Lambda

1. Introduction

Modern e-commerce systems require fast, scalable, and reliable order processing pipelines. Traditional monolithic systems struggle during peak traffic events such as flash sales, festive seasons, or large promotional events. To solve these challenges, cloud platforms such as AWS provide **serverless architectures**, which eliminate server management and scale automatically with workload.

This project demonstrates a **Serverless Order Notification & Processing System** in which:

- An order placed by a user is published to an **SNS topic**.
- SNS fan-outs the order message to **three SQS queues**.
- Each SQS queue triggers a dedicated **AWS Lambda function**:
    - **Payment Lambda** – verifies payment
    - **Inventory Lambda** – updates product stock
    - **Notification Lambda** – sends confirmation email/SMS

This architecture is fully asynchronous, event-driven, highly scalable, and resembles real-world e-commerce microservice workflows.

---

2. Necessity of the Project

E-commerce platforms must process thousands of orders per second with absolute reliability. The traditional approach requires:

- Maintaining servers
- Managing queues manually
- Scaling compute resources
- Handling failures and reprocessing

These challenges increase cost, complexity, and risk.

A serverless pipeline is needed because:

✓ It automatically scales

✓ It eliminates server management

✓ It reduces cost significantly

✓ It simplifies integration between microservices

✓ It ensures reliability with built-in retry and DLQs

This makes serverless architecture ideal for modern applications.

---

3. Motivation for the Project

The project is motivated by:

- Learning real-world cloud architectures used in Amazon, Flipkart, Myntra, Swiggy, Zomato, etc.
- Understanding how event-driven components communicate in large distributed systems.

- Reducing IT infrastructure cost using on-demand compute.
- Demonstrating how SNS + SQS + Lambda can build robust workflows.
- Preparing for cloud/DevOps job interviews.

This project aligns with cloud-native design principles and modern microservice best practices.

---

4. Objectives

✓ Automate order workflow using serverless AWS services

✓ Implement asynchronous communication using SNS and SQS

✓ Use SQS queues to decouple microservices

✓ Trigger Lambda functions for order-processing tasks

✓ Ensure reliability through retries and DLQs

✓ Observe workflow using CloudWatch logs

✓ Provide scalable and cost-efficient architecture

---

5. Literature Survey

Source / Research	Key Insights	Relevance
AWS Whitepapers on Serverless	Serverless removes infrastructure burden; autoscaling built-in.	Forms base architecture.
Messaging Queue Systems History	Queue-based systems improve reliability and decouple services.	SQS used for microservice decoupling.
Event-driven Architecture Research	Events improve efficiency and scalability in distributed systems.	SNS + SQS is the backbone of event flow.
Microservices Design Patterns	Independent services increase agility and fault tolerance.	Payment/Inventory/Notification lambdas mimic microservices.
Cloud Resilience Models	DLQs and retries prevent data loss.	Used in SQS queues here.

---

6. Research Questions

1. Can a fully serverless architecture handle real-time order processing?
2. How effective is SNS fan-out for distributing events to multiple systems?
3. Can SQS queues ensure reliability with no data loss?
4. How independently can microservices operate in asynchronous workflows?
5. Does serverless architecture reduce operational cost and complexity?

---

7. System Structure

The system consists of:

1. User / API Gateway

Sends order message to SNS topic.

2. SNS Topic – orders-topic

Broadcasts the order to multiple subscribers.

3. SQS Queues

Each queue receives a copy of the order:

- payment-queue
- inventory-queue
- notification-queue

4. AWS Lambda Functions

Triggered automatically:

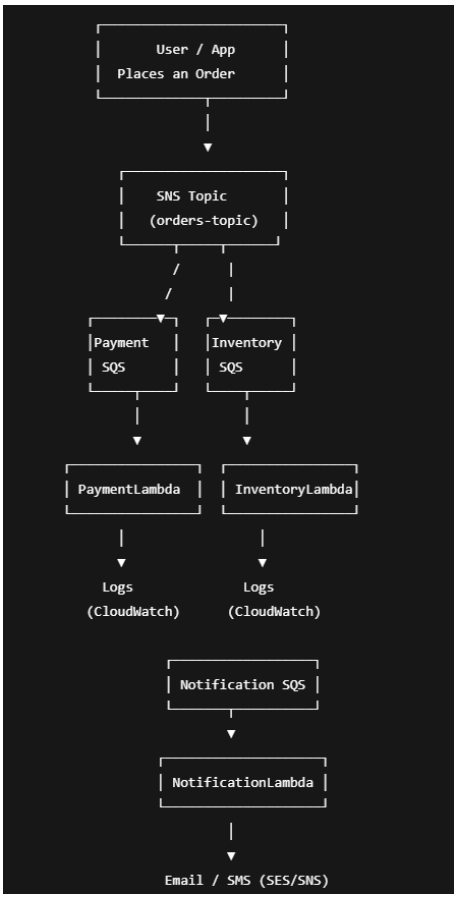
- Payment Lambda → Payment verification
- Inventory Lambda → Stock update
- Notification Lambda → Email/SMS

5. CloudWatch Logs

Captures logs from each Lambda.

---

8. System Design Framework (Architecture Diagram)



9. Methodology

Step 1 — Create SNS topic

- Name: orders-topic

Step 2 — Create SQS queues

- payment-queue, inventory-queue, notification-queue
- Attach DLQs

Step 3 — Subscribe queues to SNS topic

SNS → Subscriptions → Add SQS ARNs

Step 4 — Create IAM Roles

Add:

- AWSLambdaBasicExecutionRole
- Custom permissions for DynamoDB, SES, Secrets Manager, etc.

Step 5 — Create Lambda functions

- Add event source mapping (SQS trigger)
- Add code for each function

Step 6 — Test the workflow

Publish test message on SNS

Step 7 — Observe logs

CloudWatch → Lambda logs

Step 8 — Add monitoring

- CloudWatch alarms
- DLQ monitoring

---

10. Proposed Learning Strategy

The user learns:

- Event-driven AWS architecture
- SNS → SQS fan-out
- Lambda serverless compute
- Microservice decoupling
- Using CloudWatch for debugging
- Understanding message formats and SQS retries
- Designing reliable workflows

This enhances knowledge of modern cloud engineering practices.

---

11. Requirement Specification

Functional Requirements

- Accept order message
- Broadcast to 3 subsystems
- Process each task independently

- Send notification on success

Non-Functional Requirements

- Scalability
- Fault tolerance
- Low latency
- Zero data loss
- Cost-efficient (~few cents per 100K invocations)

Software Requirements

- AWS SNS
- AWS SQS
- AWS Lambda
- CloudWatch
- SES/SNS for notifications

Hardware Requirements

- None (serverless)

---

12. Results and Discussion

Results

- Order message was successfully published to SNS.
- All three SQS queues received the message.
- All three Lambda functions executed independently.
- Payment verification, inventory update, and notifications occurred in parallel.
- CloudWatch logs confirmed message flow end-to-end.

Discussion

- Asynchronous design improved performance.
- Zero dependency between microservices prevents failures from cascading.
- DLQs ensure no message is lost.
- Scaling is automatic — thousands of orders can be processed simultaneously.

---

13. Advantages

- Serverless (no servers to manage)
- Highly scalable
- Fault tolerant
- No data loss (SQS retries + DLQ)
- Easy debugging in CloudWatch
- Low-cost pay-per-use model
- Real-world architecture used in e-commerce

---

14. Disadvantages

- Cold starts may add slight delay

- Complex permissions for multiple services
- SES requires identity verification
- Hard to debug distributed flows without monitoring tools

---

15. Applications

- E-commerce order processing
- Payment workflow systems
- Real-time notification systems
- Inventory management
- Ticket booking systems
- Logistics and tracking systems
- Event-driven microservice architectures

---

16. Conclusion

This project demonstrates a complete **serverless e-commerce order pipeline** using AWS SNS, SQS, and Lambda. It showcases the power of event-driven architecture to build scalable, reliable, and cost-effective systems.

By decoupling components using SQS and triggering tasks via Lambda, the workflow becomes highly resilient and easy to maintain. This system reflects real-world architectures used in large companies and provides valuable hands-on cloud engineering experience.